

GoReview AI — Task Division

Project: GoReview AI — CLI architectural smell detector

Date: April 2026

Status: ~75% complete — final sprint to delivery

Overview

Five parallel tracks run simultaneously. Each person owns one complete domain from development to delivery. No one waits on anyone else to start.

#	Member	Domain	Track
1	RAHIOUI Youssef	Core Code Completion	Remaining detectors
2	NAZIH Youssef	LLM Providers + Multi-language	Anthropic · tree-sitter · TypeScript
3	Machnaoui Abdellatif	AWS Hosting + LLM Infrastructure	Lambda · Bedrock · S3
4	Hattabi Youness	UI — Extension + Dashboard	VS Code + Web dashboard
5	HOUSSY Badr	Report + Presentation	Technical report + Demo slides

Member 1 — RAHIOUI Youssef

Track: Core Code Completion

Role: Finish the remaining architectural smell detectors so the PDF detector table is 100% implemented.

Task 1.1 — Primitive Obsession Detector (Go)

File: detectors/primitiveobs/detector.go + detector_test.go

What to build: A detector that finds structs where domain concepts are typed as raw primitives instead of value objects.

Detection logic:

```
// For each struct in the parsed Go file:
// Scan all fields for types: string, int, float64, bool
// Check if field name contains domain keywords:
//   ID, Id, Email, Phone, URL, Path, Name, Code,
//   Price, Amount, Total, Count, Status, Token
// If 3+ such fields exist in one struct → emit Finding
```

```
Finding{
  Severity: core.SeverityMedium,
  SmellType: "primitive_obsession",
  Pattern:   "Value Object",
  Component: structName,
  File:      file.Path,
  Line:      structLine,
```

```

Metrics:  map[string]int{"primitive_fields": count},
Effort:   core.EffortLow,
}

```

Wire into main.go:

```

import "github.com/yourorg/gorview/detectors/primitiveobs"
// add to goDetectors slice:
primitiveobs.Detector{},

```

Tests: Write `detector_test.go` with:

- A struct with 4 primitive domain fields → expect 1 finding
- A struct with 2 primitive fields → expect 0 findings
- A struct with 3 non-domain primitive fields → expect 0 findings

Task 1.2 — Primitive Obsession Detector (Python)

File: `detectors/python/primitiveobs/detector.go` + `detector_test.go`

Same logic as 1.1 but targeting `languages/python.ParsedFile`. Detect `__init__` parameters typed or named as domain primitives.

Detection heuristic (Python has no static types, use name matching):

- Parameter names ending in `_id`, `_email`, `_phone`, `_url`, `_code`, `_name`, `_price`, `_amount`, `_token`
- Threshold: ≥ 3 such parameters in one `__init__`

Wire into main.go:

```

import pyprimobs "github.com/yourorg/gorview/detectors/python/primitiveobs"
// add to pyDs slice:
pyprimobs.Detector{},

```

Task 1.3 — Shotgun Surgery Detector (Git-based)

File: `detectors/shotgunsurgery/detector.go`

What it detects: Files that always change together across commits — a sign that a single logical change is scattered across the codebase.

Implementation (uses only `os/exec`, no new dependencies):

```

// Step 1: Get last 200 commit hashes
// git log --format="%H" -n 200
//
// Step 2: For each commit, get changed files
// git diff-tree --no-commit-id -r --name-only <hash>
//
// Step 3: Build co-change frequency map
// coChange[fileA][fileB]++ for every pair in same commit

```

```
//
// Step 4: Flag any file appearing in >5 co-change pairs
//   with frequency >= 3 → emit Finding

Finding{
    Severity: core.SeverityMedium,
    SmellType: "shotgun_surgery",
    Pattern:   "Observer / Event Bus",
    Component: fileName,
    Metrics:   map[string]int{"co_changed_with": partnerCount},
    Effort:    core.EffortHigh,
}
```

Guard: Skip gracefully if no `.git` directory present:

```
if _, err := os.Stat(".git"); os.IsNotExist(err) {
    return nil
}
```

Add flag to main.go: `--git` (default false) to enable shotgun surgery detection.

Deliverables Checklist — RAHIOUI

- ☐ `detectors/primitiveobs/detector.go`
 - ☐ `detectors/primitiveobs/detector_test.go`
 - ☐ `detectors/python/primitiveobs/detector.go`
 - ☐ `detectors/python/primitiveobs/detector_test.go`
 - ☐ `detectors/shotgunsurgery/detector.go`
 - ☐ `go test ./detectors/...` passes
 - ☐ `go build ./...` passes
-

Member 2 — NAZIH Youssef

Track: LLM Providers + Multi-language Support

Role: Add the Anthropic/Claude backend, integrate tree-sitter for universal language fallback, and add TypeScript/JS parsing.

Task 2.1 — Anthropic / Claude LLM Provider

File: `llm/anthropic/client.go`

Dependency to add:

```
go get github.com/anthropics/anthropic-sdk-go
```

Structure:

```

package anthropic

import (
    "context"
    "encoding/json"
    anthropicsdk "github.com/anthropics/anthropic-sdk-go"
    "github.com/yourorg/gorview/core"
    "github.com/yourorg/gorview/llm"
)

type Client struct {
    sdk    *anthropicsdk.Client
    model string
}

func New(apiKey, model string) llm.Provider {
    if model == "" || model == "llama3" {
        model = "claude-haiku-4-5-20251001"
    }
    return &Client{
        sdk:    anthropicsdk.NewClient(anthropicsdk.WithAPIKey(apiKey)),
        model: model,
    }
}

func (c *Client) Enrich(ctx context.Context, findings []core.Finding)
([]core.Finding, error) {
    // For each finding:
    //   prompt := llm.BuildPrompt(f)           // existing context_builder.go
    //   call Messages.New with model + prompt
    //   parse JSON response into f.Explanation, f.RefactorBefore, f.RefactorAfter,
    //   f.Effort
    // Return enriched findings
}

```

Wire into main.go `buildProvider()` :

```

case "anthropic":
    apiKey := flagLLMAPIKey
    if apiKey == "" { apiKey = os.Getenv("ANTHROPIC_API_KEY") }
    if apiKey == "" {
        return nil, fmt.Errorf("Anthropic API key required (--llm-api-key or ANTHROPIC_API_KEY)")
    }
    return anthropicLLM.New(apiKey, flagLLMModel), nil

```

Recommended model to document: `claude-haiku-4-5-20251001` — fastest, cheapest, outputs valid JSON reliably.

Task 2.2 — Tree-sitter Universal Fallback

File: `languages/treesitter/parser.go`

Dependency:

```
go get github.com/smacker/go-tree-sitter
```

What it does: Parses any source file not handled by the Go or Python parsers into a generic AST that the generic detectors can query.

```
type ParsedFile struct {
    Path      string
    Language  string // "ruby", "rust", "c", "cpp", "php", ...
    Root      *sitter.Node
    Source    []byte
}

// ParseDir scans dir for files not in alreadyParsed,
// picks the tree-sitter grammar by file extension,
// and returns ParsedFile for each supported file.
func ParseDir(dir string, alreadyParsed map[string]bool) ([]ParsedFile, error)
```

Extension → grammar map (most common):

Extension	Grammar constant
<code>.rb</code>	<code>ruby.GetLanguage()</code>
<code>.rs</code>	<code>rust.GetLanguage()</code>
<code>.c</code> , <code>.h</code>	<code>c.GetLanguage()</code>
<code>.cpp</code> , <code>.cc</code>	<code>cpp.GetLanguage()</code>
<code>.php</code>	<code>php.GetLanguage()</code>
<code>.swift</code>	<code>swift.GetLanguage()</code>

Task 2.3 — Generic Tree-sitter Detectors

File: `detectors/generic/complexity/detector.go`

Operates on `languages/treesitter.ParsedFile`. Uses S-expression node queries to count functions, branching nodes, and nesting depth — same metrics as the Go and Python complexity detectors, just sourced from the tree-sitter CST.

Task 2.4 — TypeScript / JavaScript Support

Files:

- `languages/typescript/parser.go` — uses tree-sitter-typescript grammar
- `detectors/typescript/godclass/detector.go` — class with too many methods

- `detectors/typescript/complexity/detector.go` — function with too many branches

Wire both into `main.go` the same way as the Go and Python passes.

Deliverables Checklist — NAZIH

- ☐ `llm/anthropic/client.go`
- ☐ `--llm-provider anthropic` wired into `main.go`
- ☐ `languages/treesitter/parser.go`
- ☐ `detectors/generic/complexity/detector.go`
- ☐ `languages/typescript/parser.go`
- ☐ `detectors/typescript/godclass/detector.go`
- ☐ `detectors/typescript/complexity/detector.go`
- ☐ `go build ./...` passes

Member 3 — Machnaoui Abdellatif

Track: AWS Hosting + LLM Infrastructure

Role: Deploy gorview as a serverless API on AWS, set up LLM hosting (Bedrock for cloud, Ollama on EC2 for local-style teams), and wire the CI/CD pipeline.

Task 3.1 — Lambda Wrapper

File: `cmd/lambda/main.go`

This is the entry point for the AWS Lambda function. It wraps the existing analysis logic:

```
package main

import (
    "context"
    "encoding/json"
    "fmt"
    "os"
    "os/exec"
    "github.com/aws/aws-lambda-go/lambda"
)

type Request struct {
    RepoURL string `json:"repo_url"` // public git URL to clone
    Format  string `json:"format"`    // "json" | "html"
    LLM     bool  `json:"llm"`       // enable LLM enrichment
    Provider string `json:"provider"`  // "bedrock" | "openai"
    MinScore int   `json:"min_score"` // CI threshold (0 = disabled)
}

type Response struct {
    ReportURL string `json:"report_url"` // S3 pre-signed URL
    Score     int   `json:"score"`
}
```

```

    Findings int `json:"findings"`
}

func handler(ctx context.Context, req Request) (Response, error) {
    // 1. git clone req.RepoURL into /tmp/<id>
    // 2. Build gorview flags and run analysis
    // 3. Upload JSON report to S3
    // 4. Return pre-signed URL (1-hour expiry)
}

func main() { lambda.Start(handler) }

```

New dependency:

```

go get github.com/aws/aws-lambda-go
go get github.com/aws/aws-sdk-go-v2/service/s3

```

Task 3.2 — AWS Bedrock LLM Provider

File: `llm/bedrock/client.go`

AWS Bedrock hosts Claude Haiku natively — no API key management, billed to the AWS account, no servers.

```

// Uses: github.com/aws/aws-sdk-go-v2/service/bedrockruntime
// Model ID: "anthropic.claude-haiku-4-5-20251001-v1:0"
// Request format: Anthropic Messages API (same JSON as llm/anthropic)
// Auth: IAM role attached to Lambda (no hardcoded keys)

```

Wire into `buildProvider()` as `--llm-provider bedrock`. When running inside Lambda, this is the default (no API key needed, uses the Lambda execution role).

Task 3.3 — AWS Infrastructure Deployment

Files to create: `infra/` directory

```

infra/
├─ lambda.sh      – build + deploy script
├─ policy.json    – IAM policy for Lambda role
└─ README.md     – setup instructions

```

Deployment script `infra/lambda.sh` :

```

#!/bin/bash
# Build Go binary for Lambda
GOOS=linux GOARCH=amd64 go build -o bootstrap ./cmd/lambda
zip gorview-lambda.zip bootstrap

# Deploy or update Lambda
aws lambda create-function \
    --function-name gorview-analyze \

```

```

--runtime provided.al2023 \
--handler bootstrap \
--zip-file fileb://gorview-lambda.zip \
--role arn:aws:iam::${AWS_ACCOUNT_ID}:role/gorview-lambda-role \
--timeout 300 \
--memory-size 512 \
--environment "Variables={GORVIEW_REPORT_BUCKET=gorview-reports-${AWS_ACCOUNT_ID}}" \
\
2>/dev/null || \
aws lambda update-function-code \
--function-name gorview-analyze \
--zip-file fileb://gorview-lambda.zip

```

IAM policy `infra/policy.json` :

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:PutObject", "s3:GetObject"],
      "Resource": "arn:aws:s3:::gorview-reports-*/*"
    },
    {
      "Effect": "Allow",
      "Action": ["bedrock:InvokeModel"],
      "Resource": "arn:aws:bedrock:*::foundation-model/anthropic.claude-haiku*"
    },
    {
      "Effect": "Allow",
      "Action": ["logs:CreateLogGroup", "logs:CreateLogStream",
        "logs:PutLogEvents"],
      "Resource": "*"
    }
  ]
}

```

Task 3.4 — Ollama on EC2 (Alternative for Private Teams)

For teams that want 100% private LLM without Bedrock:

Setup instructions to write in `infra/ollama-ec2.md` :

1. Launch `m7i.large` (8 GB RAM, \$0.096/hr) in the same VPC as Lambda
2. Install Ollama: `curl -fsSL https://ollama.com/install.sh | sh`
3. Pull the recommended model: `ollama pull qwen2.5-coder:7b`
4. Start as systemd service on port 11434 (VPC-internal only, no public IP)
5. Set Lambda env var: `OLLAMA_BASE_URL=http://<ec2-private-ip>:11434`

Security: Security group allows port 11434 only from the Lambda VPC security group — never public.

Task 3.5 — API Gateway

Create an HTTP API with a single route:

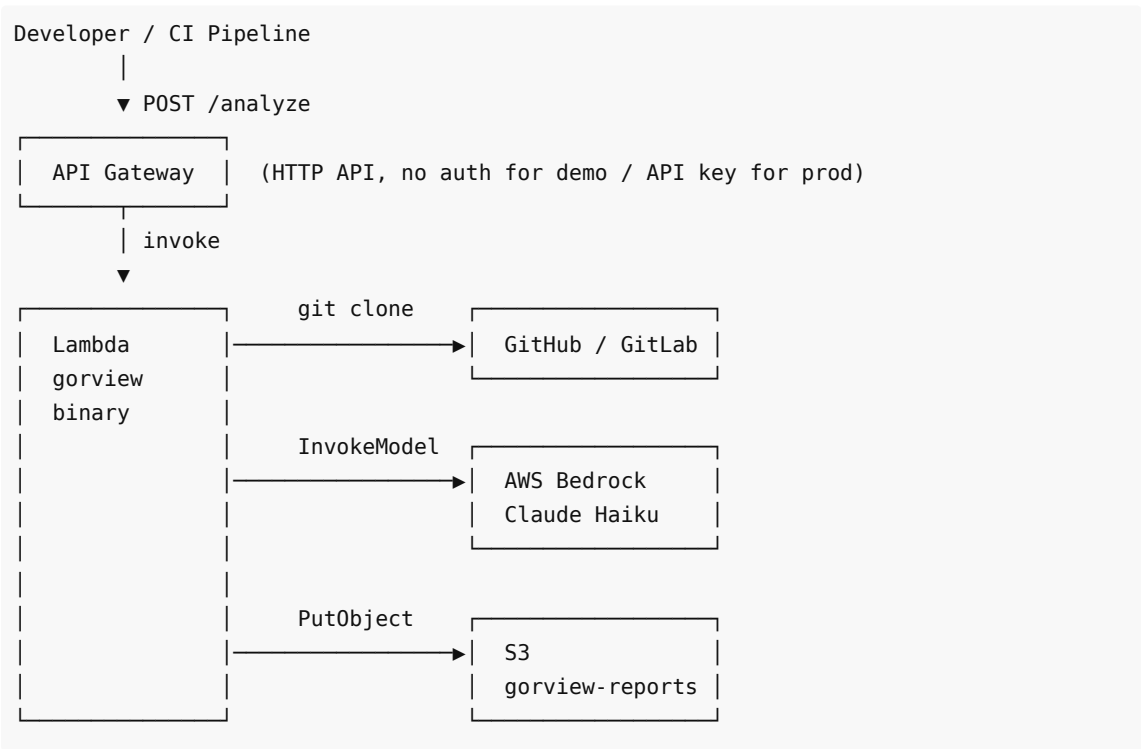
```
POST https://<api-id>.execute-api.<region>.amazonaws.com/analyze
Content-Type: application/json

{
  "repo_url": "https://github.com/yourorg/some-project",
  "format": "json",
  "llm": true,
  "provider": "bedrock"
}
```

Response:

```
{
  "report_url": "https://gorview-reports-
xxx.s3.amazonaws.com/reports/.../report.json?...",
  "score": 72,
  "findings": 5
}
```

AWS Architecture Diagram



Deliverables Checklist — Machnaoui

- ☐ cmd/lambda/main.go

- ☐ `llm/bedrock/client.go`
- ☐ `infra/lambda.sh`
- ☐ `infra/policy.json`
- ☐ `infra/ollama-ec2.md`
- ☐ Lambda function deployed and reachable via API Gateway URL
- ☐ Test: `curl -X POST <api-url> -d '{"repo_url": "..."}' -H "Content-Type: application/json"`

Member 4 — Hattabi Youness

Track: UI — VS Code Extension + Web Dashboard

Role: Build the two user-facing interfaces: a VS Code extension that runs gorview inside the editor, and a web dashboard that displays scan history and score trends.

Task 4.1 — VS Code Extension

Repository: Create a new folder `vscode-gorview/` at the project root (separate npm project).

Stack: TypeScript + VS Code Extension API

Initialization:

```
npm install -g yo generator-code
yo code
# → New Extension (TypeScript)
# → Name: gorview-ai
# → Identifier: gorview-ai
```

Key commands to implement:

Command	ID	What it does
Analyze Workspace	<code>gorview.analyzeWorkspace</code>	Runs gorview <code>./...</code> in workspace root
Analyze File	<code>gorview.analyzeFile</code>	Runs gorview on the open file's directory
Set Min Score	<code>gorview.setMinScore</code>	Prompts for threshold, saves to workspace settings

Problems Panel integration:

```
// src/extension.ts
const diagnostics = vscode.languages.createDiagnosticCollection('gorview');

async function runAnalysis(folder: string) {
  const output = await execGorview(folder, ['--format', 'json']);
  const report = JSON.parse(output);

  const byFile = new Map<string, vscode.Diagnostic[]>();
  for (const finding of report.findings) {
```

```

const diag = new vscode.Diagnostic(
  new vscode.Range(finding.line - 1, 0, finding.line - 1, 999),
  `[${finding.severity}] ${finding.smell_type} - ${finding.pattern}`,
  severityMap[finding.severity]
);
diag.source = 'GoReview AI';
// group by file
}
diagnostics.set(/* uri */, /* diags */);
}

```

Status bar item:

```

// Show "GoReview: 72/100" in the bottom status bar
const bar = vscode.window.createStatusBarItem(vscode.StatusBarAlignment.Left);
bar.text = `GoReview: ${report.score}/100`;
bar.color = report.score >= 70 ? '#4fc3f7' : '#ef5350';
bar.show();

```

Settings (package.json):

```

"configuration": {
  "gorview.binaryPath": { "type": "string", "default": "gorview" },
  "gorview.minScore": { "type": "number", "default": 0 },
  "gorview.llmProvider": { "type": "string", "enum":
["none", "ollama", "anthropic", "openai"] },
  "gorview.runOnSave": { "type": "boolean", "default": false }
}

```

Build and package:

```

npm install
npm run compile
npx vsce package # produces gorview-ai-x.x.x.vsix
# Install locally:
code --install-extension gorview-ai-x.x.x.vsix

```

Task 4.2 — Web Dashboard

Location: dashboard/ folder at the project root (standalone HTML/JS, no framework required).

Stack: Vanilla HTML + CSS + JavaScript. Calls the Lambda API (from Member 3) to fetch and display reports.

Pages / views:

Main Dashboard (index.html)

- Score gauge (SVG circle showing e.g. 72/100)
- Findings table with columns: Severity | Smell Type | File | Line | Pattern | Effort
- Filter bar: severity dropdown, smell type dropdown
- Color coding: CRITIQUE = red, MOYEN = orange, FAIBLE = yellow

History View (`history.html`)

- Line chart of score over last 10 scans (plain SVG or Chart.js CDN)
- Table: Date | Score | Findings count | Repo

Scan Form (`scan.html`)

- Input: repo URL or local path
- Checkbox: enable LLM enrichment
- Provider select: Bedrock / OpenAI / Ollama
- Submit → calls `POST /analyze` → polls until done → redirects to dashboard

API calls (`js/api.js`):

```
const API_BASE = 'https://<api-id>.execute-api.eu-west-1.amazonaws.com';

async function startScan(repoUrl, llm, provider) {
  const res = await fetch(`${API_BASE}/analyze`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ repo_url: repoUrl, llm, provider, format: 'json' })
  });
  return res.json(); // { report_url, score, findings }
}

async function loadReport(reportUrl) {
  const res = await fetch(reportUrl);
  return res.json();
}
```

Hosting: Upload `dashboard/` to S3 with static website hosting enabled:

```
aws s3 sync dashboard/ s3://gorview-dashboard-<account-id> --acl public-read
```

Or host on GitHub Pages (push `dashboard/` to a `gh-pages` branch).

Deliverables Checklist — Hattabi

- ☐ `vscode-gorview/` — compiles with `npm run compile`
 - ☐ Extension shows findings in VS Code Problems panel
 - ☐ Extension shows score in status bar
 - ☐ `gorview-ai-x.x.x.vsix` built and installable
 - ☐ `dashboard/index.html` — score gauge + findings table
 - ☐ `dashboard/history.html` — score trend chart
 - ☐ `dashboard/scan.html` — scan form
 - ☐ Dashboard deployed (S3 static site or GitHub Pages URL)
-

Member 5 — HOUSSY Badr

Track: Technical Report + Presentation

Role: Write the final technical report, build the presentation slide deck, prepare the live demo, and record a demo video.

Task 5.1 — Technical Report

Format: PDF, ~20–25 pages, academic style matching the original project proposal.

Outline:

Section	Pages	Content
Cover page	1	Title, team, date — match original proposal style
Table of contents	1	Auto-generated
1. Introduction	2	Problem statement (from PDF §1–2), why gorview, comparison table
2. Architecture	3	Package diagram, data flow, component descriptions
3. Detectors	4	One subsection per detector: what it detects, algorithm, example Finding
4. LLM Integration	2	How BuildPrompt works, privacy guarantee, provider comparison table
5. Multi-language	2	Go AST, Python walker, tree-sitter fallback strategy
6. Cloud Deployment	2	AWS architecture diagram, Lambda + Bedrock setup
7. UI & Dashboard	2	VS Code extension screenshots, dashboard screenshots
8. Results	2	Run gorview on a real open-source repo, show output, score, LLM enrichment
9. Conclusion	1	Summary, future work (v1.0 items)
References	1	Go docs, tree-sitter, Ollama, Anthropic, AWS Bedrock

Key figures to include:

- Architecture tree (from PDF §5, updated with new components)
- Terminal output screenshot (god_struct + concrete_dep findings)
- HTML report screenshot
- VS Code Problems panel screenshot
- Dashboard screenshot
- AWS architecture diagram (from Member 3)

How to get screenshots:

```
# Run gorview on a test repo with real findings:
gorview ./... 2>&1 | tee terminal_output.txt
gorview ./... -f html -o report.html
```

Task 5.2 — Presentation Slides

Tool: Any (PowerPoint, Google Slides, Canva, Beamer)

Length: 12–15 slides, 15-minute presentation + 5-minute demo

Slide structure:

Slide	Title	Content
1	Cover	GoReview AI · Team · Date
2	The Problem	Comparison table (golangci-lint vs CodeRabbit vs SonarQube vs gorview)
3	Our Solution	One-liner + terminal screenshot
4	How It Works	Two-step diagram: Static Analysis → LLM Enrichment
5	Architecture	Package tree diagram
6	Detectors	Table: smell → description → pattern (all 6 detectors)
7	Multi-language	Go AST / Python walker / tree-sitter graphic
8	LLM Providers	Ollama local + OpenAI + Anthropic + Bedrock — privacy guarantee callout
9	Cloud Deployment	AWS architecture diagram
10	VS Code Extension	Screenshot: Problems panel + status bar
11	Dashboard	Screenshot: score gauge + findings table
12	Live Demo	(placeholder — live demo happens here)
13	Results	gorview run on a real repo: score 68/100, 7 findings
14	Conclusion + Roadmap	Done vs. v1.0 items
15	Q&A	Thank you

Task 5.3 — Live Demo Script

Prepare a demo repository with intentional architectural smells. This repo should be committed to GitHub so the demo can also test the Lambda API.

Demo repo structure (`demo-smelly-shop/`):

```
// service/order.go – god struct (22 fields, 31 methods)
// handler/payment.go – concrete dependency (PaymentGateway is a struct)
// util/validator.go – high complexity (CC = 14)
// model/user.go – primitive obsession (5 raw string fields: email, phone, id, etc.)
```

Demo flow (live, 5 minutes):

1. `cd demo-smelly-shop`
2. `gorview ./...`
→ show terminal output with 4 findings, score 58/100
3. `gorview ./... --llm --llm-provider ollama --llm-model qwen2.5-coder:7b`
→ show LLM-enriched output with explanations and before/after refactoring
4. `gorview ./... -f html -o report.html && open report.html`
→ show HTML report in browser
5. Open VS Code → GoReview AI: Analyze Workspace
→ show findings appearing in Problems panel
6. Open dashboard URL → show scan history + score gauge

Task 5.4 — Demo Video (Backup)

Record a 3-minute screen capture of steps 1–4 of the demo script using OBS or any screen recorder. Upload to a private YouTube link or include in the GitHub repo as a compressed `.mp4`. This is the backup if the live demo has a technical issue.

Deliverables Checklist — HOUSSY

- ☐ Technical report PDF (20–25 pages)
 - ☐ Slide deck (12–15 slides, PDF + source)
 - ☐ `demo-smelly-shop/` repository on GitHub with README
 - ☐ Demo script document (step-by-step with expected output)
 - ☐ Demo video backup (3 min, compressed)
-
-

Shared Timeline

Week 1 – Parallel development (all 5 work independently)

RAHIOUI	primitiveobs detectors (Go + Python)
NAZIH	Anthropic provider + tree-sitter scaffold
Machnaoui	Lambda wrapper + S3 setup + Bedrock client
Hattabi	VS Code extension scaffold + commands
HOUSSY	Report outline + demo repo creation

Week 2 – Integration

RAHIOUI	Shotgun surgery detector + final tests
NAZIH	TypeScript parser + generic detectors
Machnaoui	Lambda deployed + API Gateway live
Hattabi	Dashboard pages + VS Code status bar
HOUSSY	Report sections 1–5 written

Week 3 – Polish + Demo

RAHIOUI	Code review support, fix failing tests
NAZIH	<code>go build ./...</code> clean, update <code>go.mod</code>
Machnaoui	Load test Lambda, cost estimate

Hattabi	Dashboard deployed + .vsix packaged
HOUSSY	Full report draft + slides complete
Week 4 – Final	
All	Integration test: full pipeline end-to-end
All	gorview run on a real open-source repo
HOUSSY	Final report + rehearse presentation
All	Presentation day

Integration Contract

All members must respect these interfaces so their components connect without rework:

Contract	Owner	Consumer
llm.Provider interface (unchanged)	NAZIH (Anthropic), Machnaoui (Bedrock)	main.go
core.Finding struct (unchanged)	—	All detectors (RAHIOUI), output (Hattabi dashboard)
detectors.Detector interface	RAHIOUI	main.go, HOUSSY (demo)
Lambda API POST /analyze JSON schema	Machnaoui	Hattabi (dashboard api.js)
Report JSON schema	existing output/json.go	Hattabi (dashboard), HOUSSY (demo)

Rule: No one modifies `core/finding.go` , `core/report.go` , `detectors/detector.go` , or `llm/provider.go` without notifying the group — these are the shared contracts.

Summary Table

Member	Primary Deliverable	Secondary	Effort
RAHIOUI Youssef	3 new detectors + tests	Code review	~3 files
NAZIH Youssef	Anthropic + tree-sitter + TypeScript	go.mod updates	~7 files
Machnaoui Abdellatif	Lambda + Bedrock + AWS infra	Ollama EC2 docs	~5 files + AWS
Hattabi Youness	VS Code extension + dashboard	Demo screenshots	~20 files
HOUSSY Badr	Report + slides + demo	Demo video	Documents